

Multimodal RAG Information Retrieval System

Detailed Technical Project Documentation

Project scope: Question answering over the supplied Apple Inc. Q3 2022 Form 10-Q PDF, using an evidence-first multimodal retrieval pipeline.

1. Executive Summary

The project implements a Retrieval-Augmented Generation (RAG) information-retrieval application for answering questions over Apple Inc.'s Q3 2022 Form 10-Q PDF. The system is designed to handle narrative text, financial tables and numeric disclosures, figures, charts, embedded images, and rendered page-level visual evidence.

The core design is evidence-first: answers are generated from retrieved document evidence rather than unrestricted model knowledge. The API returns supporting evidence records together with page numbers, modality labels, and relevance scores. Questions that are clearly outside the supplied filing are rejected with an explicit out-of-scope response.

- PyMuPDF-based PDF ingestion and page/image extraction.
- Transparent JSON evidence index.
- Lexical TF-IDF-style retrieval with modality-aware ranking.
- Extractive answer fallback that works without an LLM API key.
- Optional OpenAI-compatible LLM generation.
- FastAPI backend and browser-based presentation UI.
- Figure and page-image previews.
- Automated retrieval and scope regression tests.

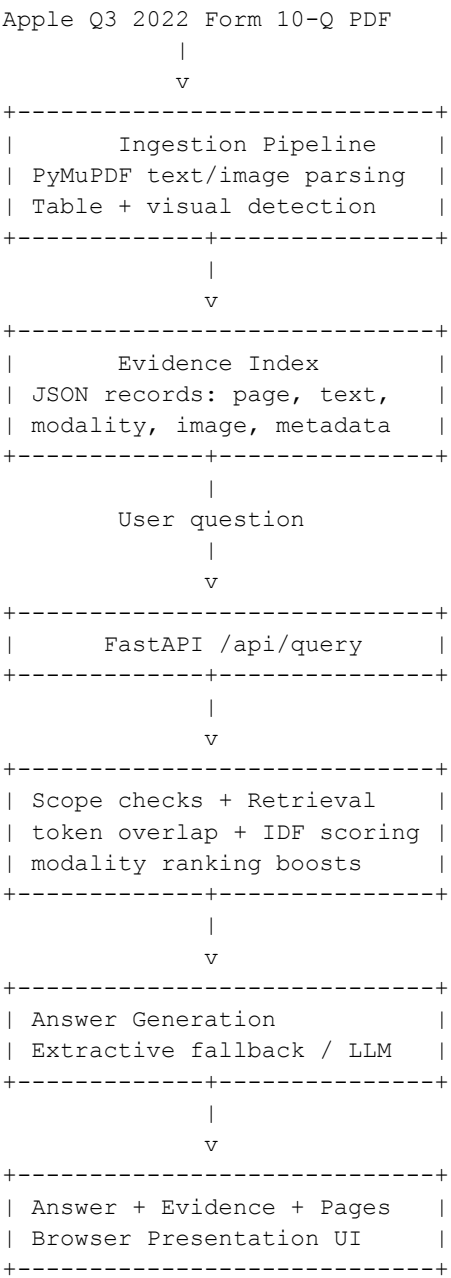
2. Project Objectives

1. Build a reproducible question-answering system around a single supplied financial filing.
2. Preserve source evidence so answers can be audited back to document pages.
3. Support multiple evidence modalities rather than relying only on plain text.
4. Provide a deterministic fallback mode when no external LLM credentials are available.
5. Prevent unsupported questions from being answered using general model knowledge.
6. Expose the retrieval layer independently through a FastAPI endpoint.
7. Provide a simple presentation-ready interface for demonstrations and technical review.

3. Scope of Supported Questions

Question type	Supported evidence	Example
Narrative text	Extracted PDF page text	Questions about disclosures or explanations in the filing
Financial tables	Table-candidate text records	Net sales, assets, liabilities, operating income
Numeric disclosures	Page text and retrieved evidence	Reported financial values
Figures/charts	Rendered page images and extracted images	Which pages contain figures or visual evidence?
Out-of-scope requests	No answer generated	Current stock price, weather, unrelated years

4. End-to-End Architecture



The implementation separates ingestion, retrieval, and generation. This modularity allows the answer-generation component to be replaced without redesigning the document ingestion pipeline.

5. Repository Structure

```
SmartDataSolutions/
├── artifacts/
│   ├── index.json           Generated evidence index
│   └── images/              Rendered pages and extracted PDF images
├── data/
│   └── aapl_2022_q3_10q.pdf  Supplied source document
├── docs/
│   └── rag_methodology.pdf   Separate methodology submission (excluded here)
├── scripts/
│   ├── ingest.py            Rebuild the document index
│   └── build_submission_pdf.py Generate methodology PDF
├── src/
│   ├── __init__.py
│   ├── app.py               FastAPI application and routes
│   └── rag/
│       ├── __init__.py
│       ├── generator.py      Answer generation
│       ├── ingest.py         PDF parsing/indexing
│       ├── models.py         Evidence data model
│       └── retriever.py      Query filtering/ranking
├── templates/
│   └── index.html           Browser UI
├── tests/
│   ├── test_retriever.py    Retrieval ranking test
│   └── test_scope.py        Out-of-scope behavior test
├── .gitignore
├── requirements.txt
└── README.md
```

The artifacts directory contains generated data. Keeping the source PDF in the data directory allows the project to be reproduced from a clean checkout.

6. Technology Stack

Layer	Technology	Responsibility
Language	Python 3.13 recommended	Application and data pipeline
PDF processing	PyMuPDF / fitz	Text extraction, image extraction, page rendering
API	FastAPI	HTTP routes, request validation, lifecycle management
ASGI server	Uvicorn	Local application server
Validation	Pydantic	Request schema and input limits
HTTP client	HTTPX	OpenAI-compatible model calls
Document index	JSON	Portable and inspectable evidence storage
UI	HTML/CSS/JavaScript	Search workspace and evidence presentation
Testing	Pytest	Retrieval and scope regression tests

The default implementation does not require a vector database, GPU, hosted model, or external service. This supports deterministic local execution and simplifies evaluation.

7. Installation and Setup

7.1 Create the virtual environment

```
cd /Users/username/Documents/SmartDataSolutions
python3 -m venv .venv
source .venv/bin/activate
```

Python 3.13 is recommended for compatibility with the pinned packages.

7.2 Install dependencies

```
pip install -r requirements.txt
```

7.3 Build or rebuild the index

```
python scripts/ingest.py
```

This reads the supplied PDF and regenerates:

```
artifacts/index.json
artifacts/images/
```

The ingestion step should be rerun whenever the source PDF changes.

7.4 Start the application

```
uvicorn src.app:app --reload

http://127.0.0.1:8000
```

The application loads the existing index at startup. If the index is missing, the FastAPI startup hook creates it automatically.

8. Optional LLM Configuration

The system can operate without an API key. In that configuration, it uses the extractive fallback implemented in the generator module.

```
export OPENAI_API_KEY="your-api-key"
export OPENAI_MODEL="gpt-4o-mini"
export OPENAI_BASE_URL="https://api.openai.com/v1"
uvicorn src.app:app --reload
```

The configured base URL may point to another OpenAI-compatible service, including a local gateway or compatible local inference endpoint. The generator sends only the user question and retrieved evidence to the configured model.

- Answer only from supplied evidence.
- Preserve dates, units, and numbers.
- Cite factual claims with page references.

- State when evidence is insufficient.
- Avoid inventing visual facts.

No API key or secret is stored in the repository.

9. PDF Ingestion and Multimodal Indexing

9.1 Text Extraction

For every PDF page, PyMuPDF extracts page text. Whitespace is normalized and the result is stored as an Evidence record.

- Stable chunk ID.
- One-based PDF page number.
- Modality label.
- Normalized text.
- Optional image path.
- Retrieval score populated during retrieval.

Large page records are capped to keep the evidence index manageable.

9.2 Table Detection

The source contains financial statements and numeric disclosures. The implementation uses conservative heuristics to label text records as table candidates when multiple signals are present.

- Multiple lines contain numeric values.
- Multiple lines appear to have column separation.
- Financial terms such as net sales, assets, liabilities, operating income, or cash flows appear.

This is a candidate detector, not a guarantee that every detected page is a perfectly reconstructed table. Original page text is retained for citation and auditing.

9.3 Visual Page Indexing

Every page is rendered to a PNG image under artifacts/images/page-N.png. Rendered page evidence acts as a visual fallback for charts, diagrams, figures, and layout-dependent content.

9.4 Embedded Image Extraction

PDF image objects are extracted individually when possible and stored with page-aware filenames such as page-1-image-1.jpeg. Each extracted image is indexed as a figure record together with its page number and nearby page text.

9.5 Evidence Record

```
{  
  "chunk_id": "p19-text",
```

```

    "page": 19,
    "modality": "table",
    "text": "Services net sales increased ...",
    "score": 0.0,
    "image_path": null,
    "metadata": {}
  }

```

JSON was selected because it is inspectable during an interview or technical review and can later be migrated to a vector store without changing the application contract.

10. Retrieval Mechanism

10.1 Query Tokenization

Questions are lowercased and tokenized using a regular expression. Common stopwords are removed to reduce noise from terms such as 'the', 'of', and 'and'.

10.2 IDF-Style Lexical Scoring

For each query token found in an evidence record, the retriever calculates a term-frequency and inverse-document-frequency contribution:

```
score += (1 + log(term_frequency)) * IDF(token)
```

This increases the contribution of terms that are relevant to the question and relatively discriminative within the document.

10.3 Modality-Aware Ranking

- Table-related terms such as revenue, income, assets, cash, or sales boost table candidates.
- Figure-related terms such as figure, chart, image, graph, shown, or pictured boost visual evidence.

The purpose is to return the appropriate type of evidence rather than simply the record with the highest lexical overlap.

10.4 Evidence Threshold

At least two meaningful query terms must match an evidence record. This reduces retrieval of unrelated paragraphs caused by a single generic word.

10.5 Out-of-Scope Protection

The retriever rejects questions that clearly request information outside the supplied filing, including:

- Current/latest/today/weather/forecast questions without document context.
- Years other than 2021 and 2022.
- Questions with no meaningful document-term overlap.

```

This question is beyond my scope. I can only answer questions supported by the supplied
Apple Q3 2022 10-Q PDF, including its text, tables, and figures.

```

This is a deliberate groundedness and user-expectation safeguard.

11. Answer Generation

11.1 Extractive Fallback

When no model API key is configured, the system:

8. Splits retrieved evidence into candidate sentences.
9. Measures overlap between question terms and each sentence.
10. Selects the highest-overlap sentences.
11. Adds the source page number to each selected sentence.

This mode is useful for offline demonstrations, reproducible evaluation, credential-free environments, and debugging retrieval independently of generation.

11.2 Optional LLM Generation

When an API key is configured, the system sends the question and retrieved evidence to the /chat/completions endpoint of the configured OpenAI-compatible service. The model does not receive the entire PDF implicitly and does not browse the internet. It receives only the evidence selected by the retriever.

This establishes the central RAG boundary: retrieval supplies grounded context, while generation converts that context into a concise answer.

11.3 Evidence and UI Separation

The answer is displayed first. Supporting evidence is placed behind a collapsible 'Supporting evidence' control, keeping the interface concise while allowing reviewers to inspect passages and visual evidence.

12. API Reference

12.1 GET /

Returns the browser application.

12.2 GET /health

Returns service status and the indexed document.

```
{
  "status": "ok",
  "indexed": true,
  "document": "aapl_2022_q3_10q.pdf"
}
```

12.3 POST /api/query

Example request:

```
{
  "question": "What were Apple's total net sales for the third quarter of 2022?",
  "limit": 6
}
```

Validation rules:

- question must contain 3–2000 characters.
- limit must be between 1 and 12.

Example response:

```
{
  "answer": "Grounded answer with page references.",
  "evidence": [
    {
      "chunk_id": "p19-text",
      "page": 19,
      "modality": "table",
      "text": "Retrieved source passage...",
      "score": 12.34,
      "image_path": null,
      "metadata": {}
    }
  ],
  "modalities": ["table"]
}
```

Example curl request:

```
curl -X POST http://127.0.0.1:8000/api/query \
-H 'content-type: application/json' \
-d '{"question": "What were Apple's net sales by product category?"}'
```

12.4 GET /api/figure/{path}

Serves an extracted or rendered image from the artifacts directory. The route resolves and validates the requested path to prevent access outside the artifact directory.

13. User Interface

The UI is implemented as a single server-rendered HTML file at templates/index.html.

- Branded document-intelligence header.
- Active source document card.
- Research workspace.
- Example question chips.
- Search button and Ctrl/Cmd + Enter shortcut.
- Loading state while retrieval runs.
- Answer card with modality badges.
- Grounded-response metadata.
- Collapsible supporting evidence.
- Page numbers and relevance scores.

- Inline figure and page-image previews.
- Responsive layout.

The browser does not perform retrieval itself. It sends JSON to `/api/query` and renders the returned response, keeping the browser layer thin and the API independently testable.

14. Testing and Validation

```
source .venv/bin/activate
python -m pytest -q
```

14.1 Retrieval Ranking Test

`tests/test_retriever.py` verifies that a record containing relevant terms is ranked above an unrelated record.

14.2 Scope Protection Test

`tests/test_scope.py` verifies that an unrelated weather question returns no evidence and produces the expected out-of-scope message.

14.3 Additional Manual Validation

```
python -m compileall -q src scripts
python scripts/ingest.py
python -m pytest -q
uvicorn src.app:app --reload
```

15. Assumptions

12. The supplied PDF is digitally generated and contains extractable text.
13. The filing context is limited to the 2021 and 2022 periods represented in the document.
14. A page containing numeric, column-like financial text is a useful table candidate even when semantic table structure is unavailable.
15. A rendered page is a valid visual-evidence fallback for figures and charts.
16. Users benefit more from a grounded limitation message than from an unsupported answer.

16. Known Limitations

16.1 Table Reconstruction

The implementation identifies table candidates but does not reconstruct every table into a fully structured row/column representation. PDF layout extraction varies substantially across documents.

16.2 Figure Understanding

Figures and page images are indexed and displayed, but the default extractive mode does not perform detailed visual reasoning over image pixels. A multimodal vision-language model would be required for questions involving chart trends or diagram relationships.

16.3 Retrieval Quality

The baseline retriever uses sparse lexical retrieval. It does not currently use dense embeddings, semantic reranking, query expansion, or a vector database.

16.4 Numeric Reasoning

Numeric evidence is retrieved, but there is no dedicated financial calculation layer. Arithmetic questions should be supported by a verified calculation tool that cites source values before computing.

16.5 Production Security

The local demonstration does not provide authentication, rate limiting, user accounts, or persistent query history. These controls should be introduced before public deployment.

17. Future Improvements

17. Replace heuristic table detection with layout-aware table extraction.
18. Add OCR for scanned pages and image-only PDFs.
19. Add dense embeddings and hybrid sparse/dense retrieval.
20. Add a cross-encoder reranker for top retrieved candidates.
21. Store embeddings and metadata in a vector database.
22. Add image embeddings and vision-language evidence analysis.
23. Add a calculation tool for percentages, comparisons, and financial totals.
24. Add answer-faithfulness and citation-completeness evaluation.
25. Add caching for repeated questions.
26. Add authentication, rate limiting, logging, and monitoring.
27. Add a document-upload workflow for multiple PDFs.
28. Add configurable chunking instead of one primary text record per page.

18. Quick Demonstration

```
cd /Users/username/Documents/SmartDataSolutions
source .venv/bin/activate
python scripts/ingest.py
python -m pytest -q
uvicorn src.app:app --reload
```

Open <http://127.0.0.1:8000> and test questions such as:

What were Apple's total net sales for the third quarter of 2022?

What were Apple's net sales by product category?

Which pages contain figures or visual evidence?

Negative-scope test:

What is Apple's current stock price?

The negative-scope question is expected to return the documented out-of-scope message instead of an unsupported current-value answer.